

A Method for Contour Location of Industrial Objects Based on CNN

Jiangyun Li and Qian Zhang

School of Automation and Electrical Engineering, University of Science and Technology Beijing
Key Laboratory of Advanced Control of Iron and Steel Process (Ministry of Education)
Beijing, China

Emails: jiangyunlee@gmail.com; zdhzq0323@163.com

Abstract—There are many industrial fields need to locate the specific texture or contour of the particular objects in the image with complex scenes. Edge detection algorithm based on gray transform is generally hard to handle these issues. This paper presented a model that can accurately locate the specific texture or contour in RGB images with complex scenes by modifying and improving the CNN-based edge detection model HED. It described and practiced the improving method in detail, using accurately locating the outer contour of the plate in images as an example. It achieves the target effectively and accurately and is proved to be a method for only locating the specific texture in images.

Keywords—CNN; HED; contour location; specific texture

I. INTRODUCTION

In industrial applications, there are many scenes relying heavily on machine vision technology to extract the specific texture or contour in images. Because there always exists strong interference items which are not in the focus areas but have the very similar characteristics. We usually need to accurately separate the focus and non-focus area, or try hard to minimize the impact that the non-focus areas make. Sometimes, we also need to accurately outline the specific boundary in the image for measurement.

Marr mentioned in Theory of edge detection [1]: Edge is the junction between one area and another, the place where including the most concentrated information and mutation property. Though light changes can significantly affect the appearance of the area, they cannot change the edge. The traditional edge detection methods based on gray transformation, including Sobel detector [2] and Canny detector [3], on the one hand, are largely affected by changes in light and environment, so they perform not good in the detection and identification of complex scenes. On the other hand, it is very difficult for them to identify a particular texture or contour which is more important in many applications. The edge detection methods based on careful manual design, including Statistical Edges [4], gPb [5], Sketch Tokens [6], are very dependent on the wealth of expertise, and it is very complex to design the top features to fit all kinds of complex scenes.

Many algorithms based on CNN have made remarkable achievements in image recognition in recent years [7], even surpassed human-level performance on ImageNet classification. They can be trained to learn and abstract the top features all by

themselves that is their unique advantage. So some researchers began to use them to solve edge location and extraction problems, and developed many good algorithms including HED [8], DeepEdge [9], DeepContour [10]. HED is a very superior one, short for holistically-nested edge detection. It leverages fully convolutional neural networks and deeply-supervised nets to address two important issues: holistic image training and prediction, multi-scale and multi-level feature learning. It can automatically learn rich hierarchical representations, and reach the state-of-the-art on the BSDS500 dataset [5] and the NYU Depth dataset [11]. For a 320*480 image, it costs only 0.4s predicting the final edge map which is magnitude faster than some other CNN-based edge detection algorithms.

Although there are many edge detection algorithms, very few excellent results have been made for the location of specific texture or contour, which is very important in many industrial applications. This paper develops a method to locate the specific texture or contour in RGB images of any size, inspired by the HED algorithm, which contributes new ideas for industrial applications to solve the related issues.

II. NETWORK ARCHITECTURE

CNN is a supervised deep learning algorithm, typically including training, validation and testing three processes. Training process is a gradually iterative process to find the optimal model which can produce similar predictions satisfying the labeling requirements. It includes the process of forward propagation, which can produce the prediction for the input image based on the existing network parameters, and back propagation, which can make the network to change parameters relying on the prediction and the optimization algorithm set up previously. Validation process can produce the predictions for validation dataset in order to validate the performance of the network, usually carried out in the training process. Testing process is used to fully test the network to obtain the recognition accuracy and other related indicators.

Fully convolutional network [12] modifies the networks for recognition to generate a prediction for each pixel with only one loss function. It cannot produce accurately multi-scale prediction, but edge detection not only requires predictions for every pixel, but also needs locate the edge pixels accurately. FCN preforms not good in this aspect [8], but inspires the production of HED model. HED deeply supervises the network

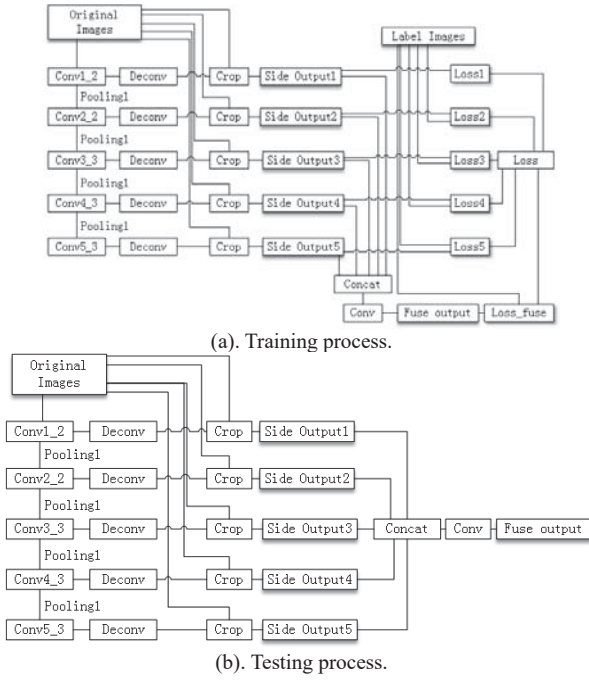


Fig. 1. The training and testing processes for HED. (a). In the training process: The input is the original images and the label images. The last convolution layer in every stage produces a feature map in the same size as original image through deconvolution. Five feature maps crop with the original images, then produce the side-output. Five side-outputs produce one fusion output through fusion layer. Six outputs all compare with the label and get six loss, then the combined loss affects the iterative process; (b). In the testing process, the input is the original images, and the output is five side-output predictions and one fusion prediction.

using method demonstrated in the deeply-supervised net approach [13] to supervise the hidden layers on the basis of FCN.

From the year of 2012, more and more deep enough CNN models with more than one stage emerged in the ImageNet large Scale Visual Recognition Competition [7], [14]. These models usually contain a lot of parameters which leads to very difficult training from scratch. So many CNN models for detecting usually use the basis parameters of networks [8], [12] performing excellent results in ILSVRC, then they will expand the network, modify the optimization algorithm and adjust the parameters. VGGNet [15] achieved excellent results in ILSVRC2014, winning first place in the location contest and second place in the recognition contest. It has sufficient depth (16 or 19 layers of convolution), extremely dense convolution(convolution kernels:3, stride:1) and five stages.

HED modifies on the basis of VGGNet16 that retains all the network structures before the fifth pooling layer and uses its parameters as the initial arguments to adjust the network. In the meanwhile, it connects the last convolution layer in every stage to a side-output layer that gets a feature map in the same size as original image through deconvolution, then it connects the five side-outputs to the fusion layer to get a fusion output. In each iteration of the training process, six outputs all compare with the label image to compute loss. They all affect the parameter changes in back propagation thus forming the underlying outputs and top output monitoring the final results together. In the test process, the final output of HED is the average result of the five side-outputs and the fusion output.



Fig. 2. Examples of training dataset.

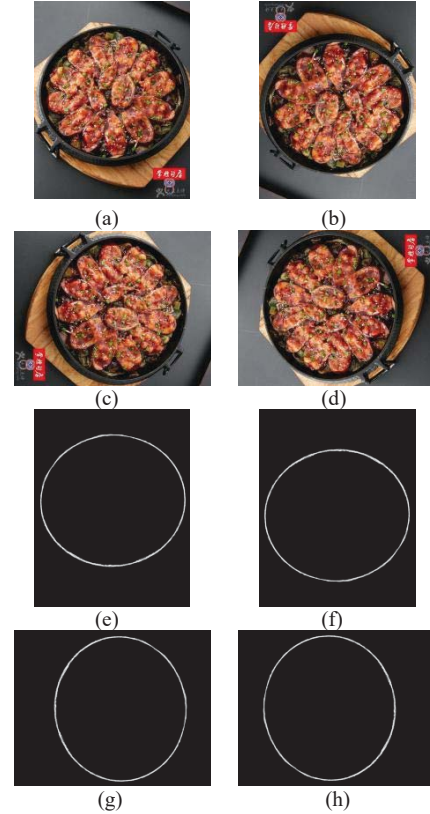


Fig. 3. Examples of dataset creation and expansion. (a). the original image. (b). the original image rotated 180. (c). the original image rotated 90. (d). the original image rotated 270. (e). the label image. (f). the label image rotated 180. (g). the label image rotated 90. (h). the label image rotated 270.

III. MODIFY THE NETWORK

HED model is a typical network model training under supervise which use the label image to guide the network self-learning the features that edges have. The inputs of the training process for this model are the original images and their corresponding label images. With the increase of the iterations, the network will tend to produce the predictions similar to the label requiring. After a lot of iterations, the model will be used to detect the coherence edge with a certain degree of accuracy.

For a specific object, its texture or contour characteristic is always close, such as all the outer contour of the steel before crimping and cutting, all the outer contour of the carton, all the

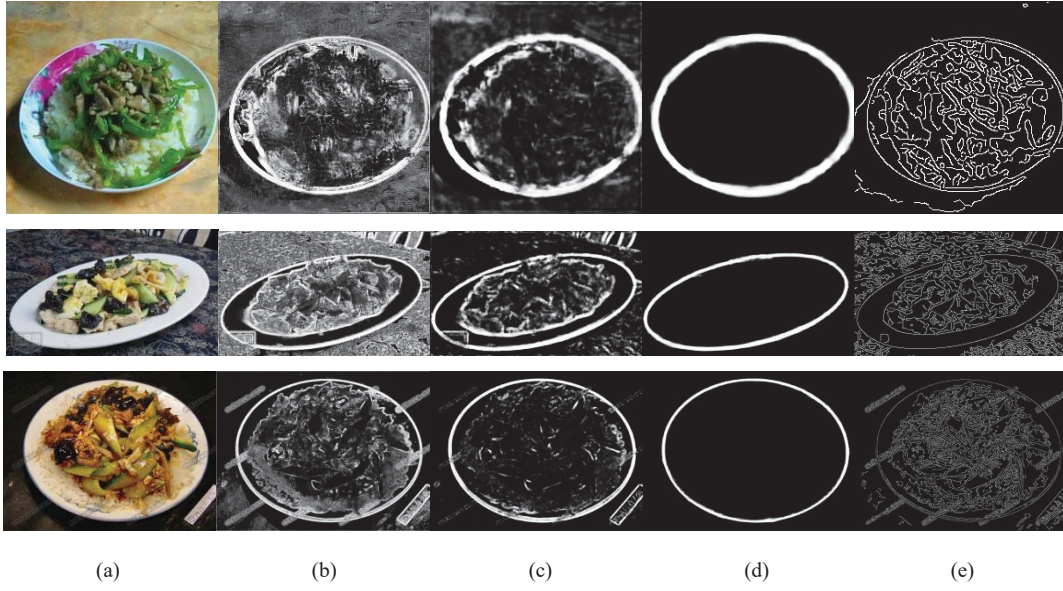


Fig. 4. Examples of the contour located 100%. (a). The original image. (b). The first side-output. (c). The second side-output. (d). The fusion output. (e). The output produced by Canny.

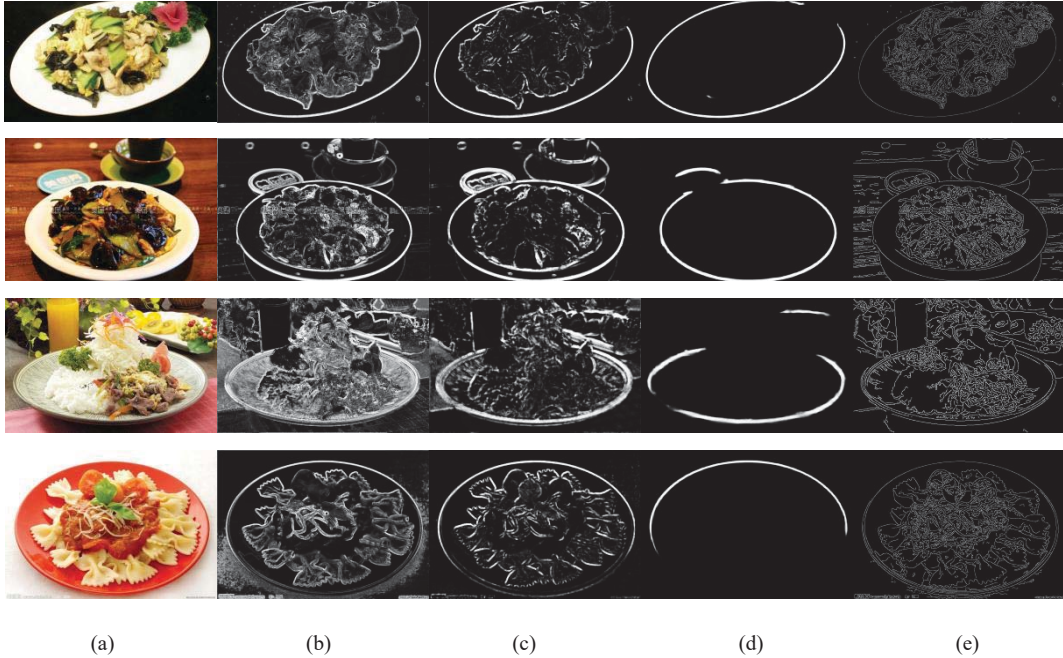


Fig. 5. Examples of the contour located less than 100%. (a). The original image. (b). The first side-output. (c). The second side-output. (d). The fusion output. (e). The output produced by Canny.

texture of the plate and so on. So we can only mark the specific texture or contour with some specific values as the label image to train the model again with different optimization algorithms. With the increase of the iterations, the network will tend to produce the predictions detecting the specific texture or contour of these specific objects. After a lot of iterations, the model will be used to locate the specific texture or contour with a certain degree of accuracy. This paper produces a model that can locate specific texture or contour of a specific object in industrial applications by modifying and adjusting the HED network according to these ideas. The following is a detailed description for the creation and expansion of the dataset and the network

parameters adjustment.

A. Dataset Creation and Expansion

There is no standard dataset in this purpose, and most images for the major image datasets come from Internet. This paper is set to locate the outer contour of the plate in images and gets images that meet the requirements from Internet to create a training and testing dataset. There are only three requirements when collecting the images: (1) In addition to the plate, there should be complex background and food in plate; (2) Different shapes of the plates; (3) The plate appears fully in the image.

There are 300 training images in BSDS500 dataset. We

collect 250 images meeting the requirements as the training dataset. But the dataset is too small, and it is very difficult to create the label images, so we rotate the original images 90/180/270 to expand the dataset. When creating the label images, we only mark the outer contour of the plates with gray label and the other pixels are all black. We create 1000 training images in total.

B. Network Parameters Adjustment

We use the deep learning framework Caffe [16] completing the training and testing process to get the HED model. We change the learning rate to $1e-7$, and make each iteration only enter a pair of images. Then we use the training dataset we create as input images to train the HED model. After lots of iterations, the model gets the ability to locate the specific contour accurately.

IV. RESULT

We collect 50 images as test samples, which are not all similar with the training images in order to know the factors that have greater impact. Then we use the model produced after ten thousandth iteration completing the testing process. The contour is located 100% in 26 images, more than 90% in 15 images, more than 70% but less than 90% in 5 images, less than 70% in only 4 images.

The testing results show that our model really can locate the fully outer contour of the plate and performs very well. All size of the images that meet the requirements can become the input of our model. But the bigger the image is, the slower the model will produce the prediction and the more memory will be taken up.

In Figs. 4 and 5, we can make the following conclusion: (1) The first side-output of our model tend to extract all the edge of the input image. (2) The second side-output of our model gets more abstract and clear results than the first one. (3) The fusion output can achieve the goal to locate the specific texture or contour, but is very rough. (4) The output produced by Canny detector is very fine and contains a lot of textures. (5) The Canny detector is affected greatly by light and environmental changes. (6) Our model is affected by the scenes change, if the scenes are in the training data, then the performance will be good. (7) Our model can deal with RGB images directly, but Canny detector only can deal with gray images.

The current model produced in this paper has several problems to be improved: (1) The prediction from our model is very rough, which is related with the deconvolution algorithm directly using bilinear interpolation algorithm enlarging a small feature map to the size of the original image. (2) When detecting the occluded edges, our model performs relatively poor. We do not give the clear label under this situation in the training dataset, because it all depends on the project demands. (3) Our model can only mask parts of the curved textures, shown in Fig. 5. (4) When facing the simple scenes, our model performs relatively poor, shown in Fig. 5.

Though our model still has some drawbacks, but we can enhance the accurate rate by increasing the training dataset with more scenes, or modifying the optimization algorithm for this network.

Our model can successfully predict the position of the outer contour of the plate in images, and complete predicting 50 testing images with different sizes in 20s on the K40 server. In the meanwhile, our model design ideas are also suitable for many scenes which need to extract the specific texture of a specific object.

V. CONCLUSION

This paper produces a model can successfully locate the outer contour of the plate based on HED model. It can make end-to-end detection, accepts random size of input image, realizes the hidden layer supervision, has fast test speed and can ignore the texture or contour in non-focus areas. Our model design ideas are also suitable for many scenes which need to extract the specific texture of a specific object.

REFERENCES

- [1] D. Marr and E. Hildreth, "Theory of edge detection," *Proc. Roy. Soc. Lond.*, vol. 207, pp. 187-217, 1980.
- [2] I. Sobel, "Camera models and machine perception," Artificial Intelligence Project, *Stanford Univ., Stanford, CA, Memo AIM-121*, 1970.
- [3] J. Canny, "A Computational Approach to Edge Detection," *IEEE Trans. Pattern Anal. Machine Intell.*, vol. 8, pp. 679-698, 2010.
- [4] S. Konishi, A. L. Yuille, J. M. Coughlan, S. C. Zhu, "Statistical edge detection: learning and evaluating edge cues," *IEEE Trans. Pattern Anal. Machine Intell.*, vol. 25, pp. 57-74, 2003.
- [5] P. Arbelaez, M. Maire, C. Fowlkes, and J. Malik, "Contour detection and hierarchical image segmentation," *IEEE Trans. Pattern Anal. Machine Intell.*, vol. 33, pp. 898-916, 2011.
- [6] J. J. Lim, C. L. Zitnick, and P. Dollar, "Sketch Tokens: A Learned Mid-level Representation for Contour and Object Detection," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Portland*, pp. 3158-3165, 2013.
- [7] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, pp. 436-444, 2015.
- [8] S. Xie and Z. Tu, "Holistically-Nested Edge Detection," *IEEE International Conference on Computer Vision, Santiago*, pp. 1395-1403, 2015.
- [9] G. Bertasius, J. Shi, and L. Torresani, "DeepEdge: A multi-scale bifurcated deep network for top-down contour detection" *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Boston*, pp. 4380-4389, 2015.
- [10] W. Shen, X. Wang, Y. Wang, X. Bai, and Z. Zhang, "DeepContour: A deep convolutional feature learned by positive-sharing loss for contour detection," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Boston*, pp. 3982-3991, 2015.
- [11] N. Silberman, D. Hoiem, P. Kohli, and R. Fergus, "Indoor Segmentation and Support Inference from RGBD Images," *European Conference on Computer Vision, Florence*, pp. 746-760, 2012.
- [12] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Boston*, pp. 3431-3440, 2015.
- [13] C. Y. Lee, S. Xie, P. Gallagher, Z. Zhang, and Z. Tu, "Deeply-Supervised Nets," *International Conference on Artificial Intelligence and Statistics, San Diego*, pp. 6, 2015.
- [14] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," *Neural Information Processing Systems Conference, Lake Tahoe*, pp. 1097-1105, 2012.
- [15] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition" *arXiv preprint arXiv:1409.1556*, 2014.
- [16] Y. Jia, E. Shelhamer, J. Donahue, S. Karayev, J. Long, R. Girshick, et al., "Caffe: Convolutional architecture for fast feature embedding," *Proceedings of the 22nd ACM international conference on Multimedia, Orlando*, pp. 675-678, 2014.